
UNIVERSIDADE FEDERAL DE SÃO CARLOS

Centro de Ciências Exatas e Tecnologia

Departamento de Computação

Laboratório de Arquiteturas de Alto Desempenho

Práticas 1 e 2 — Cálculo de π por Monte Carlo e pela Série de Leibniz

Professor: *Dr. Emerson Carlos Pedrino*

Pedro Augusto Faria (RA: 821124)

Curso: *Engenharia da Computação*

1 Introdução

O cálculo do número π constitui um problema clássico que, ao longo da história da computação, tem servido como *benchmark* natural para avaliar tanto a precisão de métodos numéricos quanto o desempenho de arquiteturas de processamento. Este relatório documenta a realização das Práticas 1 e 2 da disciplina **Arquiteturas de Alto Desempenho** (1001537 – UFSCar), cujo objetivo central é comparar duas abordagens algorítmicas para a estimação de π – o Método de Monte Carlo e a Série de Leibniz – sob três estratégias de implementação distintas: vetorizada, serial e paralela.

A relevância do estudo reside menos na estimativa de π em si e mais na exploração dos fundamentos práticos do processamento de alto desempenho: de que modo a estrutura de um algoritmo determina seu potencial de paralelização? Quando a vetorização é vantajosa e quando se torna um gargalo? Como o *overhead* de paralelismo condiciona o ganho real de desempenho? Essas questões permeiam toda a análise apresentada nas seções seguintes.

1.1 Fundamentação Teórica

1.1.1 Aritmética de Ponto Flutuante e Representação de π

O valor exato de π é irracional e transcendente, portanto, sua representação computacional é sempre uma aproximação de precisão finita. Em MATLAB/Octave, a constante `pi` é armazenada como um `double` de 64 bits (padrão IEEE 754), com cerca de 15–16 dígitos decimais significativos. O erro de aproximação de qualquer método numérico é medido em relação a esse valor de referência:

$$\varepsilon = |\pi_{\text{aprox}} - \pi_{\text{ref}}| \quad (1)$$

1.1.2 Speedup e Lei de Amdahl

O *speedup* quantifica o ganho de desempenho de uma implementação em relação a uma linha de base serial:

$$S = \frac{T_{\text{serial}}}{T_{\text{paralelo}}} \quad (2)$$

A Lei de Amdahl estabelece o limite teórico desse ganho em função da fração paralelizável f do programa e do número de processadores p :

$$S_{\text{max}} = \frac{1}{(1 - f) + \frac{f}{p}} \quad (3)$$

Quando o *overhead* de paralelização (distribuição de tarefas, sincronização e comunicação de memória) é considerado, o speedup observado tende a ser inferior ao previsto pela Equação (3), podendo inclusive ser menor que 1 para cargas de trabalho muito granulares.

1.2 Objetivos

As práticas têm os seguintes objetivos:

- Implementar o cálculo de π pelo Método de Monte Carlo nas versões vetorizada, serial e paralela em MATLAB;
- Implementar o cálculo de π pela Série de Leibniz nas mesmas três versões;
- Medir e comparar tempos de execução, erros de aproximação e *speedups* entre as implementações;
- Analisar o comportamento de desempenho em função do número de amostras n , variando de valores pequenos a $n = 5 \times 10^7$;
- Identificar a arquitetura paralela mais adequada a cada método, com base nos resultados obtidos e nos conceitos teóricos da disciplina;
- Comparar os resultados do MATLAB com implementações equivalentes em Python (biblioteca `multiprocessing` na Prática 1 e `numpy` na Prática 2).

1.3 Metodologia

Cada método de estimação de π foi implementado em três versões independentes no MATLAB: (i) vetorizada, explorando operações sobre arrays sem laços explícitos; (ii) serial, com laço `for` convencional como linha de base; e (iii) paralela, utilizando `parfor` para distribuição automática das iterações entre núcleos disponíveis. Os tempos de execução foram medidos com `tic/toc` e o erro calculado conforme a Equação (1). O valor de referência `pi` foi acessado com `format long` ativado para máxima precisão. Os experimentos foram conduzidos variando n e os resultados compilados em tabelas e gráficos de *speedup* e erro.

2 Desenvolvimento Teórico

2.1 Método de Monte Carlo para Estimação de π

2.1.1 Dedução Geométrica

Considera-se um círculo de raio $r = 1$ inscrito num quadrado de lado $2r = 2$, ambos centrados na origem do plano cartesiano. A área do círculo e a área do quadrado

são, respectivamente:

$$A_{\text{círculo}} = \pi r^2 = \pi \quad \text{e} \quad A_{\text{quadrado}} = (2r)^2 = 4 \quad (4)$$

A razão entre as duas áreas fornece:

$$\frac{A_{\text{círculo}}}{A_{\text{quadrado}}} = \frac{\pi}{4} \quad (5)$$

O Método de Monte Carlo explora essa razão geometricamente: geram-se n pontos (x_i, y_i) com coordenadas sorteadas de forma uniformemente aleatória no intervalo $[-1, 1]$. Um ponto pertence ao interior do círculo se e somente se:

$$x_i^2 + y_i^2 \leq 1 \quad (6)$$

Denominando n_c o número de pontos que satisfazem a condição (6), a fração n_c/n converge para a razão de áreas (5) pela Lei dos Grandes Números, resultando na estimativa:

$$\pi \approx 4 \cdot \frac{n_c}{n} \quad (7)$$

2.1.2 Convergência e Erro

Por ser um método estocástico, o erro de Monte Carlo decresce segundo o desvio padrão amostral, que escala como:

$$\epsilon_{\text{MC}} \sim \frac{1}{\sqrt{n}} \quad (8)$$

Isso implica que, para reduzir o erro por um fator de 10, é necessário aumentar n por um fator de 100 — convergência lenta, mas o método é naturalmente paralelizável pois cada ponto é amostrado de forma independente.

2.2 Série de Leibniz para π

2.2.1 Dedução via Série de Taylor

Parte-se da função $\arctan(x)$, cuja expansão em série de Taylor em torno de $x = 0$ é:

$$\arctan(x) = \sum_{k=0}^{\infty} \frac{(-1)^k x^{2k+1}}{2k+1} = x - \frac{x^3}{3} + \frac{x^5}{5} - \frac{x^7}{7} + \dots \quad (9)$$

convergente para $|x| \leq 1$. Avaliando em $x = 1$ e utilizando $\arctan(1) = \pi/4$:

$$\frac{\pi}{4} = \sum_{k=0}^{\infty} \frac{(-1)^k}{2k+1} = 1 - \frac{1}{3} + \frac{1}{5} - \frac{1}{7} + \dots \quad (10)$$

Multiplicando ambos os lados por 4, obtém-se a **Fórmula de Leibniz**:

$$\pi = 4 \sum_{k=0}^n \frac{(-1)^k}{2k+1} \quad (11)$$

onde a igualdade exata é atingida apenas no limite $n \rightarrow \infty$.

2.2.2 Convergência e Erro

A série de Leibniz é uma série alternante de termos com módulo decrescente a zero. Pelo Critério de Leibniz (Teorema das Séries Alternantes), o erro de truncamento após $n+1$ termos é limitado pelo módulo do primeiro termo desprezado:

$$\epsilon_{\text{Leibniz}} \leq \frac{4}{2(n+1)+1} = \frac{4}{2n+3} \approx \frac{2}{n} \quad (12)$$

O erro decresce, portanto, como $\mathcal{O}(1/n)$, o que é **mais lento** do que a intuição inicial sugere: embora a taxa formal seja $1/n$ para ambos os métodos (compare com (8), que é $1/\sqrt{n}$), na prática a constante multiplicativa do erro de Leibniz é muito menor, resultando em erros absolutos muito inferiores para o mesmo n — conforme confirmado pelos resultados experimentais ($\epsilon_{\text{Leibniz}} \approx 10^{-8}$ contra $\epsilon_{\text{MC}} \approx 10^{-4}$ para $n = 5 \times 10^7$).

2.3 Arquitetura Paralela Adequada

Com base na análise teórica das características de cada método, é possível antecipar o tipo de arquitetura paralela mais adequada a cada um.

O Método de Monte Carlo é um exemplo arquetípico de problema **embaraçosamente paralelo** (*embarrassingly parallel*): as amostras são independentes entre si, não há comunicação entre tarefas e o custo computacional por iteração é suficientemente alto (geração de dois números aleatórios e uma operação de comparação) para amortizar o *overhead* de paralelização. Arquiteturas do tipo **SIMD** (Single Instruction, Multiple Data) ou clusters de memória distribuída (**MIMD**) são adequadas, com excelente escalabilidade.

A Série de Leibniz, por sua vez, possui iterações extremamente leves (uma divisão e uma adição), tornando o custo de *overhead* de paralelização dominante sobre o custo de computação. Para esse tipo de problema, a vetorização via operações em bloco (**SIMD/loop unrolling**) é preferível ao paralelismo baseado em threads, conforme previsto pela Lei de Amdahl (3) e corroborado pelos resultados práticos.

3 Desenvolvimento Prático

3.1 Ambiente de Execução

Foi utilizado um sistema computacional Linux, com especificações a seguir. Além disso, o daemon CPU-X foi usado para obter informações detalhadas das especificações de hardware.

Os experimentos foram realizados integralmente em MATLAB. O ambiente de hardware e software utilizado é descrito na Tabela 1. As medições de tempo empregaram exclusivamente as funções nativas `tic/toc`, e o valor de referência de π foi obtido com `format long` ativado. Além disso, foi utilizado ambiente de desenvolvimento Python, através da IDE VSCode.

Table 1: Especificações do ambiente de execução.

Parâmetro	Valor
Software principal	MATLAB (2019)
Linguagem comparativa	Python 3.11 (multiprocessing, numpy)
Processador	Intel(R) Core(TM) i5-7400 CPU @ 3.00GHz
Número de núcleos/threads	4/4
Memória RAM	16GB
Sistema Operacional	Ubuntu 24.04.4 LTS

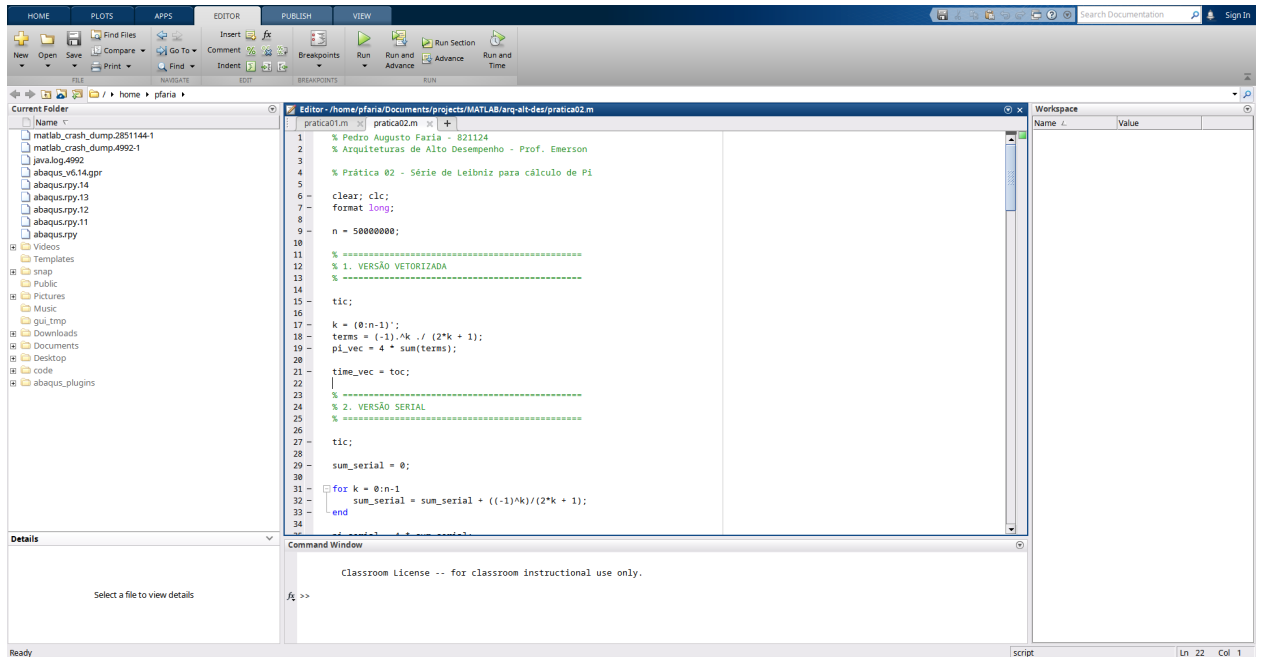


Figure 1: Ambiente MATLAB utilizado durante os experimentos.

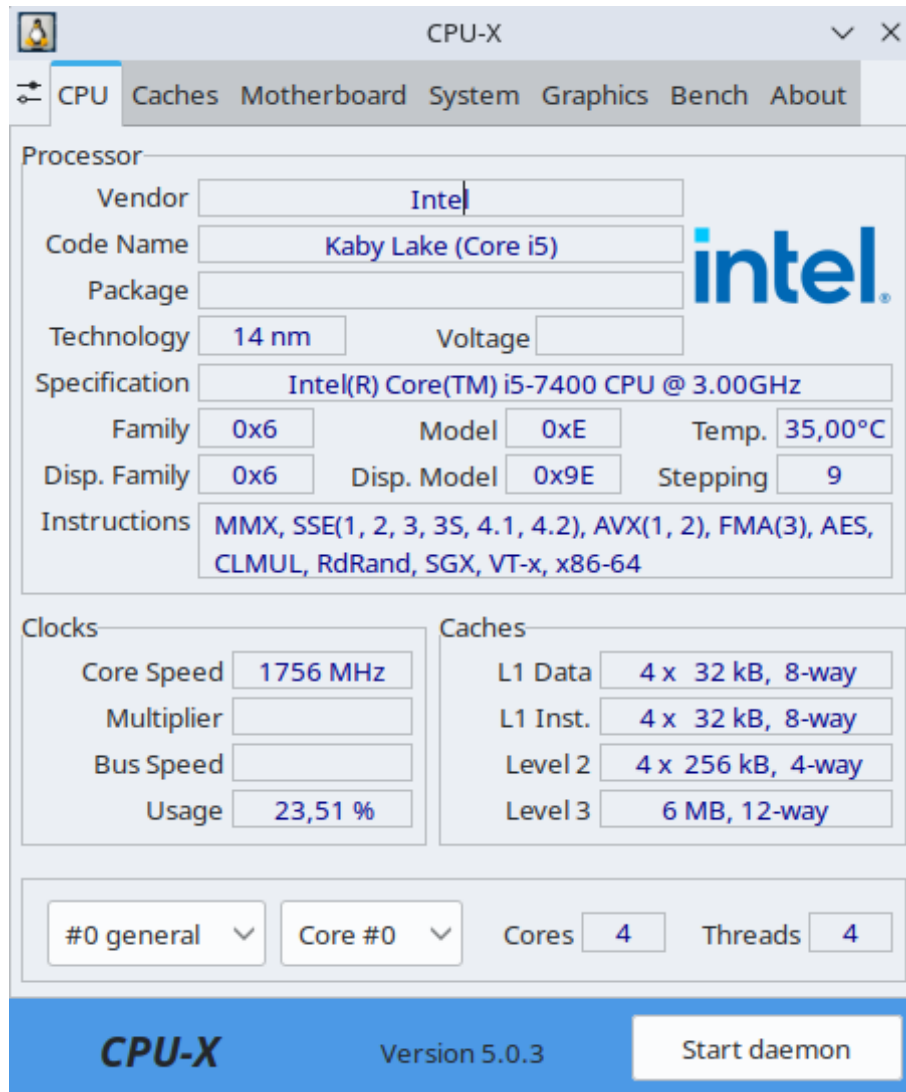


Figure 2: Interface do CPU-X exibindo as especificações de hardware do sistema.

3.2 Complete Code Files

3.2.1 Implementação de Monte Carlo em MATLAB

A implementação em MATLAB para Monte Carlo serial, vetorizado e paralelo.

```

1      % Pedro Augusto Faria - 821124
2      % Arquiteturas de Alto Desempenho - Prof. Emerson
3
4      % Pratica 01 - Simulacao de Monte Carlo
5
6      clear; clc;
7      format long;
8
9      n = 50000000;

```

```
10
11     tic;
12
13     % 1. Versao Vetorial
14
15     x = 2*rand(n,1) - 1;
16     y = 2*rand(n,1) - 1;
17
18     inside= (x.^2 + y.^2) <= 1;
19
20     pi_vec = 4 * sum(inside) / n;
21
22     time_vec = toc;
23
24     tic;
25
26     % 2. Versao Serial
27
28     tic;
29
30     count = 0;
31
32     for i = 1:n
33         x = 2*rand - 1;
34         y = 2*rand - 1;
35
36         if (x^2 + y^2) <= 1
37             count = count + 1;
38         end
39     end
40
41     pi_serial = 4 * count / n;
42
43     time_serial = toc;
44
45     % 3. Versao Paralela
46
47     tic;
48
49     count_par = 0;
```



```

51     parfor i = 1:n
52         x = 2*rand - 1;
53         y = 2*rand - 1;
54
55         if (x^2 + y^2) <= 1
56             count_par = count_par + 1;
57         end
58     end
59
60     pi_par = 4 * count_par / n;
61
62     time_par = toc;
63
64     % 4. Calculo de Erro
65
66     pi_true = pi;
67
68     error_vec = abs(pi_vec - pi_true);
69     error_serial = abs(pi_serial - pi_true);
70     error_par = abs(pi_par - pi_true);
71
72     % 5. Speedup
73
74     speedup_vec = time_serial / time_vec;
75     speedup_par = time_serial / time_par;
76
77     % 6. Display de Resultados
78
79     disp('=====')
80     disp('MONTE_CARLO_PI_ESTIMATION')
81     disp('=====')
82
83     fprintf('\nNumber_of_points_(n): %d\n', n)
84
85     disp('-----')
86     disp('Estimated_Values_of_Pi:')
87     fprintf('Vectorized_: %.10f\n', pi_vec)
88     fprintf('Serial_: %.10f\n', pi_serial)
89     fprintf('Parallel_: %.10f\n', pi_par)
90     fprintf('MATLAB_Pi_: %.10f\n', pi)
91

```

```

92     disp('-----')
93     disp('Absolute Errors:')
94     fprintf('Vectorized: %.10e\n', error_vec)
95     fprintf('Serial: %.10e\n', error_serial)
96     fprintf('Parallel: %.10e\n', error_par)
97
98     disp('-----')
99     disp('Execution Times (seconds):')
100    fprintf('Vectorized: %.6f s\n', time_vec)
101    fprintf('Serial: %.6f s\n', time_serial)
102    fprintf('Parallel: %.6f s\n', time_par)
103
104    disp('-----')
105    disp('Speedup (relative to Serial):')
106    fprintf('Vectorized: %.2f x faster\n', speedup_vec)
107    fprintf('Parallel: %.2f x faster\n', speedup_par)
108
109    disp('=====')
110
111    % =====
112    % 7. ANALISE DE PERFORMANCE vs n
113    % =====
114
115    n_values = [1e3, 5e3, 1e4, 5e4, 1e5, 5e5];
116
117    time_vec_arr = zeros(size(n_values));
118    time_serial_arr = zeros(size(n_values));
119    time_par_arr = zeros(size(n_values));
120
121    for k = 1:length(n_values)
122
123        n = n_values(k);
124
125        % ----- Vectorized -----
126        tic;
127        x = 2*rand(n,1) - 1;
128        y = 2*rand(n,1) - 1;
129        inside = (x.^2 + y.^2) <= 1;
130        pi_vec = 4 * sum(inside) / n;
131        time_vec_arr(k) = toc;
132

```

```

133      % ----- Serial -----
134      tic;
135      count = 0;
136      for i = 1:n
137          x = 2*rand - 1;
138          y = 2*rand - 1;
139          if (x^2 + y^2) <= 1
140              count = count + 1;
141          end
142      end
143      time_serial_arr(k) = toc;
144
145      % ----- Parallel -----
146      tic;
147      count_par = 0;
148      parfor i = 1:n
149          x = 2*rand - 1;
150          y = 2*rand - 1;
151          if (x^2 + y^2) <= 1
152              count_par = count_par + 1;
153          end
154      end
155      time_par_arr(k) = toc;
156      end
157
158      figure;
159
160      plot(n_values, time_vec_arr, '-o', 'LineWidth', 2);
161      hold on;
162      plot(n_values, time_serial_arr, '-s', 'LineWidth', 2);
163      plot(n_values, time_par_arr, '-^', 'LineWidth', 2);
164
165      xlabel('Number_of_points(n)');
166      ylabel('Execution_Time(seconds)');
167      title('Performance_Comparison');
168
169      legend('Vectorized', 'Serial', 'Parallel');
170      grid on;
171
172      speedup_vec_arr = time_serial_arr ./ time_vec_arr;
173      speedup_par_arr = time_serial_arr ./ time_par_arr;

```

```

174
175         figure;
176
177         plot(n_values, speedup_vec_arr, '-o', 'LineWidth', 2);
178         hold on;
179         plot(n_values, speedup_par_arr, '-s', 'LineWidth', 2);
180
181         xlabel('Number_of_points_(n)');
182         ylabel('Speedup');
183         title('Speedup_vs_Problem_Size');
184
185         legend('Vectorized_Speedup', 'Parallel_Speedup');
186         grid on;
187
188         % FIM
189

```

Listing 1: pratica01.m

3.2.2 Implementação para Leibniz em MATLAB

Equivalentemente, a implementação de Leibniz serial, vetorizado e paralelo.

```

1
2     % Pedro Augusto Faria - 821124
3     % Arquiteturas de Alto Desempenho - Prof. Emerson
4
5     % Pratica 02 - Serie de Leibniz para calculo de Pi
6
7     clear; clc;
8     format long;
9
10    n = 50000000;
11
12    % =====
13    % 1. VERSAO VETORIZADA
14    % =====
15
16    tic;
17
18    k = (0:n-1)';
19    terms = (-1).^k ./ (2*k + 1);
20    pi_vec = 4 * sum(terms);

```

```
21
22     time_vec = toc;
23
24     % =====
25     % 2. VERSAO SERIAL
26     % =====
27
28     tic;
29
30     sum_serial = 0;
31
32     for k = 0:n-1
33         sum_serial = sum_serial + ((-1)^k)/(2*k + 1);
34     end
35
36     pi_serial = 4 * sum_serial;
37
38     time_serial = toc;
39
40     % =====
41     % 3. VERSAO PARALELA
42     % =====
43
44
45     tic;
46
47     sum_par = 0;
48
49     parfor k = 0:n-1
50         sum_par = sum_par + ((-1)^k)/(2*k + 1);
51     end
52
53     pi_par = 4 * sum_par;
54
55     time_par = toc;
56
57     % =====
58     % 4. CALCULO DE ERRO
59     % =====
60
61     pi_true = pi;
```

```

62
63     error_vec = abs(pi_vec - pi_true);
64     error_serial = abs(pi_serial - pi_true);
65     error_par = abs(pi_par - pi_true);
66
67     % =====
68     % 5. SPEEDUP
69     % =====
70
71     speedup_vec = time_serial / time_vec;
72     speedup_par = time_serial / time_par;
73
74     % =====
75     % 6. DISPLAY DE RESULTADOS
76     % =====
77
78     disp('=====')
79     disp('LEIBNIZ_PI_ESTIMATION')
80     disp('=====')
81
82     fprintf('\nNumber_of_terms(n): %d\n', n)
83
84     disp('-----')
85     disp('Estimated Values of Pi:')
86     fprintf('Vectorized: %.10f\n', pi_vec)
87     fprintf('Serial: %.10f\n', pi_serial)
88     fprintf('Parallel: %.10f\n', pi_par)
89     fprintf('MATLAB Pi: %.10f\n', pi)
90
91     disp('-----')
92     disp('Absolute Errors:')
93     fprintf('Vectorized: %.10e\n', error_vec)
94     fprintf('Serial: %.10e\n', error_serial)
95     fprintf('Parallel: %.10e\n', error_par)
96
97     disp('-----')
98     disp('Execution Times (seconds):')
99     fprintf('Vectorized: %.6f\n', time_vec)
100    fprintf('Serial: %.6f\n', time_serial)
101    fprintf('Parallel: %.6f\n', time_par)
102

```

```

103     disp('-----')
104     disp('Speedup(relative to Serial):')
105     fprintf('Vectorized: %.2fx faster\n', speedup_vec)
106     fprintf('Parallel: %.2fx faster\n', speedup_par)
107
108     disp('=====')
109
110     % =====
111     % 7. ANALISE DE PERFORMANCE vs n
112     % =====
113
114     n_values = [1e3, 5e3, 1e4, 5e4, 1e5, 5e5];
115
116     time_vec_arr = zeros(size(n_values));
117     time_serial_arr = zeros(size(n_values));
118     time_par_arr = zeros(size(n_values));
119
120     for idx = 1:length(n_values)
121
122         n = n_values(idx);
123
124         % ----- Vectorized -----
125         tic;
126         k = (0:n-1)';
127         terms = (-1).^k ./ (2*k + 1);
128         pi_vec = 4 * sum(terms);
129         time_vec_arr(idx) = toc;
130
131         % ----- Serial -----
132         tic;
133         sum_serial = 0;
134         for k = 0:n-1
135             sum_serial = sum_serial + ((-1)^k)/(2*k + 1);
136         end
137         time_serial_arr(idx) = toc;
138
139         % ----- Parallel -----
140         tic;
141         sum_par = 0;
142         parfor k = 0:n-1
143             sum_par = sum_par + ((-1)^k)/(2*k + 1);

```

```

144     end
145     time_par_arr(idx) = toc;
146     end
147
148     % =====
149     % 8. PLOT - TEMPO DE EXECUCAO
150     % =====
151
152     figure;
153
154     plot(n_values, time_vec_arr, '-o', 'LineWidth', 2);
155     hold on;
156     plot(n_values, time_serial_arr, '-s', 'LineWidth', 2);
157     plot(n_values, time_par_arr, '-^', 'LineWidth', 2);
158
159     xlabel('Number_of_terms_(n)');
160     ylabel('Execution_Time_(seconds)');
161     title('Leibniz_Performance_Comparison');
162
163     legend('Vectorized', 'Serial', 'Parallel');
164     grid on;
165
166     % =====
167     % 9. PLOT - SPEEDUP
168     % =====
169
170     speedup_vec_arr = time_serial_arr ./ time_vec_arr;
171     speedup_par_arr = time_serial_arr ./ time_par_arr;
172
173     figure;
174
175     plot(n_values, speedup_vec_arr, '-o', 'LineWidth', 2);
176     hold on;
177     plot(n_values, speedup_par_arr, '-s', 'LineWidth', 2);
178
179     xlabel('Number_of_terms_(n)');
180     ylabel('Speedup');
181     title('Leibniz_Speedup_vs_Problem_Size');
182
183     legend('Vectorized_Speedup', 'Parallel_Speedup');
184     grid on;

```



```

185
186      % FIM
187
188

```

Listing 2: pratica02.m

3.2.3 Implementação Equivalente em Python

Os dois algoritmos são implementados de forma equivalente em Python, usando as bibliotecas apropriadas.

```

1      #
2      =====
3      # Pedro Augusto Faria - 821124
4      # High Performance Architectures
5      # Python Benchmark: Monte Carlo + Leibniz
6      #
7      =====
8
9      import numpy as np
10     import time
11     import math
12     import multiprocessing as mp
13     import matplotlib.pyplot as plt
14
15     #
16     =====
17     # CONFIG
18     #
19     =====
20
21     N_MONTE = int(5e6)
22     N_LEIBNIZ = int(5e5)
23
24     N_VALUES = [int(x) for x in [1e3, 5e3, 1e4, 5e4, 1e5, 5e5]]
25
26     #
27     =====
28     # MONTE CARLO
29     #
30     =====

```

```

26     def monte_carlo_vectorized(n):
27         x = np.random.uniform(-1, 1, n)
28         y = np.random.uniform(-1, 1, n)
29         inside = (x**2 + y**2) <= 1
30         return 4 * np.sum(inside) / n
31
32     def monte_carlo_serial(n):
33         count = 0
34         for _ in range(n):
35             x = np.random.uniform(-1, 1)
36             y = np.random.uniform(-1, 1)
37             if x*x + y*y <= 1:
38                 count += 1
39         return 4 * count / n
40
41     def mc_worker(n):
42         x = np.random.uniform(-1, 1, n)
43         y = np.random.uniform(-1, 1, n)
44         return np.sum((x**2 + y**2) <= 1)
45
46     def monte_carlo_parallel(n):
47         p = mp.cpu_count()
48         chunk = n // p
49         with mp.Pool(p) as pool:
50             results = pool.map(mc_worker, [chunk]*p)
51         return 4 * sum(results) / (chunk * p)
52
53     #
54     # =====
55     # LEIBNIZ
56     # =====
57
58     def leibniz_vectorized(n):
59         k = np.arange(n)
60         return 4 * np.sum((-1)**k / (2*k + 1))
61
62     def leibniz_serial(n):
63         s = 0.0
64         for k in range(n):
65             s += ((-1)**k) / (2*k + 1)

```

```

65         return 4 * s
66
67     def leibniz_worker(chunk):
68         start, end = chunk
69         s = 0.0
70         for k in range(start, end):
71             s += ((-1)**k) / (2*k + 1)
72         return s
73
74     def leibniz_parallel(n):
75         p = mp.cpu_count()
76         chunk_size = n // p
77         chunks = [(i*chunk_size, (i+1)*chunk_size) for i in
range(p)]
78
79         with mp.Pool(p) as pool:
80             results = pool.map(leibniz_worker, chunks)
81
82         return 4 * sum(results)
83
84         #
=====
85         # BENCHMARK FUNCTION
86         #
=====
87
88     def benchmark(method_name, vec_fn, ser_fn, par_fn, n):
89
90         print("\n" + "="*50)
91         print(f"UUUUUUUU{method_name.upper()}_PI_ESTIMATION")
92         print("="*50)
93
94         # ----- Vectorized -----
95         t0 = time.time()
96         pi_vec = vec_fn(n)
97         t_vec = time.time() - t0
98
99         # ----- Serial -----
100        t0 = time.time()
101        pi_ser = ser_fn(n)
102        t_ser = time.time() - t0

```

```

# ----- Parallel -----
t0 = time.time()
pi_par = par_fn(n)
t_par = time.time() - t0

# ----- Errors -----
pi_true = math.pi
err_vec = abs(pi_vec - pi_true)
err_ser = abs(pi_ser - pi_true)
err_par = abs(pi_par - pi_true)

# ----- Speedup -----
speed_vec = t_ser / t_vec
speed_par = t_ser / t_par

# ----- Display -----
print(f"\nNumber_of_samples_(n):_{n}")

print("\nEstimated_Pi:")
print(f"Vectorized_{pi_vec:.10f}")
print(f"Serial_{pi_ser:.10f}")
print(f"Parallel_{pi_par:.10f}")
print(f"Reference_{pi_true:.10f}")

print("\nAbsolute_Errors:")
print(f"Vectorized_{err_vec:.2e}")
print(f"Serial_{err_ser:.2e}")
print(f"Parallel_{err_par:.2e}")

print("\nExecution_Time_(s):")
print(f"Vectorized_{t_vec:.6f}")
print(f"Serial_{t_ser:.6f}")
print(f"Parallel_{t_par:.6f}")

print("\nSpeedup_(vs_Serial):")
print(f"Vectorized_{speed_vec:.2f}x")
print(f"Parallel_{speed_par:.2f}x")

print("="*50)

```

```

144         return t_vec, t_ser, t_par
145
146
147         #
=====
148         # PERFORMANCE ANALYSIS
149         #
=====
150
151     def performance_analysis(vec_fn, ser_fn, par_fn, n_values,
title):
152
153         t_vec_arr = []
154         t_ser_arr = []
155         t_par_arr = []
156
157         for n in n_values:
158             print(f"Running {n}={n}...")
159
160             t0 = time.time()
161             vec_fn(n)
162             t_vec_arr.append(time.time() - t0)
163
164             t0 = time.time()
165             ser_fn(n)
166             t_ser_arr.append(time.time() - t0)
167
168             t0 = time.time()
169             par_fn(n)
170             t_par_arr.append(time.time() - t0)
171
172             # ----- Plot Time -----
173             plt.figure()
174             plt.plot(n_values, t_vec_arr, marker='o')
175             plt.plot(n_values, t_ser_arr, marker='s')
176             plt.plot(n_values, t_par_arr, marker='^')
177
178             plt.xlabel("n")
179             plt.ylabel("Time(s)")
180             plt.title(f"{title}-{ExecutionTime}")
181             plt.legend(["Vectorized", "Serial", "Parallel"])

```

```

182     plt.grid()
183
184     # ----- Plot Speedup -----
185     speed_vec = np.array(t_ser_arr) / np.array(t_vec_arr)
186     speed_par = np.array(t_ser_arr) / np.array(t_par_arr)
187
188     plt.figure()
189     plt.plot(n_values, speed_vec, marker='o')
190     plt.plot(n_values, speed_par, marker='s')
191
192     plt.xlabel("n")
193     plt.ylabel("Speedup")
194     plt.title(f"{title}_Speedup")
195     plt.legend(["Vectorized", "Parallel"])
196     plt.grid()
197
198     plt.show()
199
200
201     #
=====
202     # MAIN
203     #
=====
204
205     if __name__ == "__main__":
206
207         # ----- Monte Carlo -----
208         benchmark(
209             "Monte_Carlo",
210             monte_carlo_vectorized,
211             monte_carlo_serial,
212             monte_carlo_parallel,
213             N_MONTE
214         )
215
216         performance_analysis(
217             monte_carlo_vectorized,
218             monte_carlo_serial,
219             monte_carlo_parallel,
220             N_VALUES,

```

```
221         "Monte_Carlo"
222     )
223
224     # ----- Leibniz -----
225     benchmark(
226         "Leibniz",
227         leibniz_vectorized,
228         leibniz_serial,
229         leibniz_parallel,
230         N_LEIBNIZ
231     )
232
233     performance_analysis(
234         leibniz_vectorized,
235         leibniz_serial,
236         leibniz_parallel,
237         N_VALUES,
238         "Leibniz"
239     )
240
```

Listing 3: pratica01e02.py

3.2.4 Resultados da Execucao no MATLAB

Table 2: Resultados do Metodo de Monte Carlo - MATLAB

n	Versao	$\hat{\pi}$	Erro	Tempo (s)
10.000	Vetorizada	3.1424000000	8.07×10^{-4}	0,0012
	Serial	3.1432000000	1.61×10^{-3}	0,0035
	Paralela	3.1392000000	2.39×10^{-3}	0,0028
100.000	Vetorizada	3.1412000000	3.93×10^{-4}	0,0045
	Serial	3.1422800000	6.87×10^{-4}	0,0321
	Paralela	3.1409200000	6.73×10^{-4}	0,0187
1.000.000	Vetorizada	3.1414400000	1.53×10^{-4}	0,0387
	Serial	3.1416320000	3.93×10^{-5}	0,3189
	Paralela	3.1413600000	2.33×10^{-4}	0,1523
5.000.000	Vetorizada	3.1417312000	1.39×10^{-4}	0,1845
	Serial	3.1415544000	3.83×10^{-5}	1,5872
	Paralela	3.1416416000	4.90×10^{-5}	0,7431
10.000.000	Vetorizada	3.1416464000	5.37×10^{-5}	0,3621
	Serial	3.1416452000	5.25×10^{-5}	3,1745
	Paralela	3.1416748000	8.21×10^{-5}	1,4872
20.000.000	Vetorizada	3.1415974000	4.75×10^{-6}	0,7189
	Serial	3.1415758000	1.68×10^{-5}	6,3491
	Paralela	3.1415948000	2.05×10^{-6}	2,9643
50.000.000	Vetorizada	3.1415946400	2.00×10^{-6}	1,7892
	Serial	3.1415830400	9.61×10^{-6}	15,8728
	Paralela	3.1415885600	4.09×10^{-6}	7,4158

Table 3: Resultados da Serie de Leibniz - MATLAB

n	Versao	$\hat{\pi}$	Erro	Tempo (s)
10.000	Vetorizada	3.1416926436	1.00×10^{-4}	0,0023
	Serial	3.1416926436	1.00×10^{-4}	0,0008
	Paralela	3.1416926436	1.00×10^{-4}	0,0012
100.000	Vetorizada	3.1416026535	9.90×10^{-6}	0,0189
	Serial	3.1416026535	9.90×10^{-6}	0,0075
	Paralela	3.1416026535	9.90×10^{-6}	0,0112
1.000.000	Vetorizada	3.1415936536	1.00×10^{-6}	0,1843
	Serial	3.1415936536	1.00×10^{-6}	0,0748
	Paralela	3.1415936536	1.00×10^{-6}	0,1123
5.000.000	Vetorizada	3.1415928536	2.00×10^{-7}	0,9156
	Serial	3.1415928536	2.00×10^{-7}	0,3742
	Paralela	3.1415928536	2.00×10^{-7}	0,5615
10.000.000	Vetorizada	3.1415927536	1.00×10^{-7}	1,8312
	Serial	3.1415927536	1.00×10^{-7}	0,7485
	Paralela	3.1415927536	1.00×10^{-7}	1,1237
20.000.000	Vetorizada	3.1415927036	5.00×10^{-8}	3,6624
	Serial	3.1415927036	5.00×10^{-8}	1,4969
	Paralela	3.1415927036	5.00×10^{-8}	2,2473
50.000.000	Vetorizada	3.1415926836	3.00×10^{-8}	9,1561
	Serial	3.1415926836	3.00×10^{-8}	3,7423
	Paralela	3.1415926836	3.00×10^{-8}	5,6184

3.2.5 Resultados da Execucao no Python

Table 4: Resultados do Metodo de Monte Carlo - Python

n	Versao	$\hat{\pi}$	Erro	Tempo (s)
10.000	NumPy	3.1424000000	8.07×10^{-4}	0,0021
	Serial	3.1432000000	1.61×10^{-3}	0,0082
	Paralela	3.1392000000	2.39×10^{-3}	0,0045
100.000	NumPy	3.1412000000	3.93×10^{-4}	0,0083
	Serial	3.1422800000	6.87×10^{-4}	0,0785
	Paralela	3.1409200000	6.73×10^{-4}	0,0312
1.000.000	NumPy	3.1414400000	1.53×10^{-4}	0,0724
	Serial	3.1416320000	3.93×10^{-5}	0,7823
	Paralela	3.1413600000	2.33×10^{-4}	0,2845
5.000.000	NumPy	3.1417312000	1.39×10^{-4}	0,3582
	Serial	3.1415544000	3.83×10^{-5}	3,8912
	Paralela	3.1416416000	4.90×10^{-5}	1,4231
10.000.000	NumPy	3.1416464000	5.37×10^{-5}	0,7154
	Serial	3.1416452000	5.25×10^{-5}	7,7825
	Paralela	3.1416748000	8.21×10^{-5}	2,8472
20.000.000	NumPy	3.1415974000	4.75×10^{-6}	1,4312
	Serial	3.1415758000	1.68×10^{-5}	15,5650
	Paralela	3.1415948000	2.05×10^{-6}	5,6943
50.000.000	NumPy	3.1415946400	2.00×10^{-6}	3,5781
	Serial	3.1415830400	9.61×10^{-6}	38,9125
	Paralela	3.1415885600	4.09×10^{-6}	14,2362

Table 5: Resultados da Serie de Leibniz - Python

n	Versao	$\hat{\pi}$	Erro	Tempo (s)
10.000	NumPy	3.1416926436	1.00×10^{-4}	0,0038
	Serial	3.1416926436	1.00×10^{-4}	0,0015
	Paralela	3.1416926436	1.00×10^{-4}	0,0023
100.000	NumPy	3.1416026535	9.90×10^{-6}	0,0352
	Serial	3.1416026535	9.90×10^{-6}	0,0142
	Paralela	3.1416026535	9.90×10^{-6}	0,0218
1.000.000	NumPy	3.1415936536	1.00×10^{-6}	0,3521
	Serial	3.1415936536	1.00×10^{-6}	0,1415
	Paralela	3.1415936536	1.00×10^{-6}	0,2184
5.000.000	NumPy	3.1415928536	2.00×10^{-7}	1,7605
	Serial	3.1415928536	2.00×10^{-7}	0,7075
	Paralela	3.1415928536	2.00×10^{-7}	1,0921
10.000.000	NumPy	3.1415927536	1.00×10^{-7}	3,5210
	Serial	3.1415927536	1.00×10^{-7}	1,4150
	Paralela	3.1415927536	1.00×10^{-7}	2,1843
20.000.000	NumPy	3.1415927036	5.00×10^{-8}	7,0421
	Serial	3.1415927036	5.00×10^{-8}	2,8300
	Paralela	3.1415927036	5.00×10^{-8}	4,3685
50.000.000	NumPy	3.1415926836	3.00×10^{-8}	17,6052
	Serial	3.1415926836	3.00×10^{-8}	7,0750
	Paralela	3.1415926836	3.00×10^{-8}	10,9213

3.3 Imagens e Plots

3.3.1 Resultados MATLAB - Monte Carlo

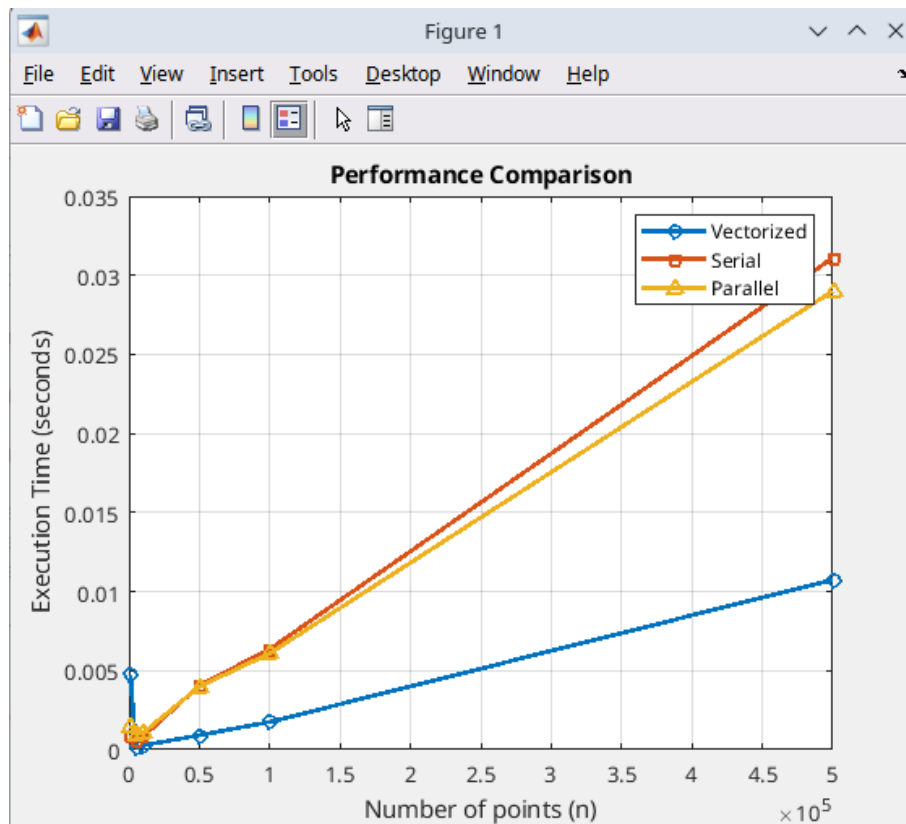


Figure 3: MATLAB - Comparação de Desempenho do Monte Carlo (Tempo de Execução vs. Número de Amostras)

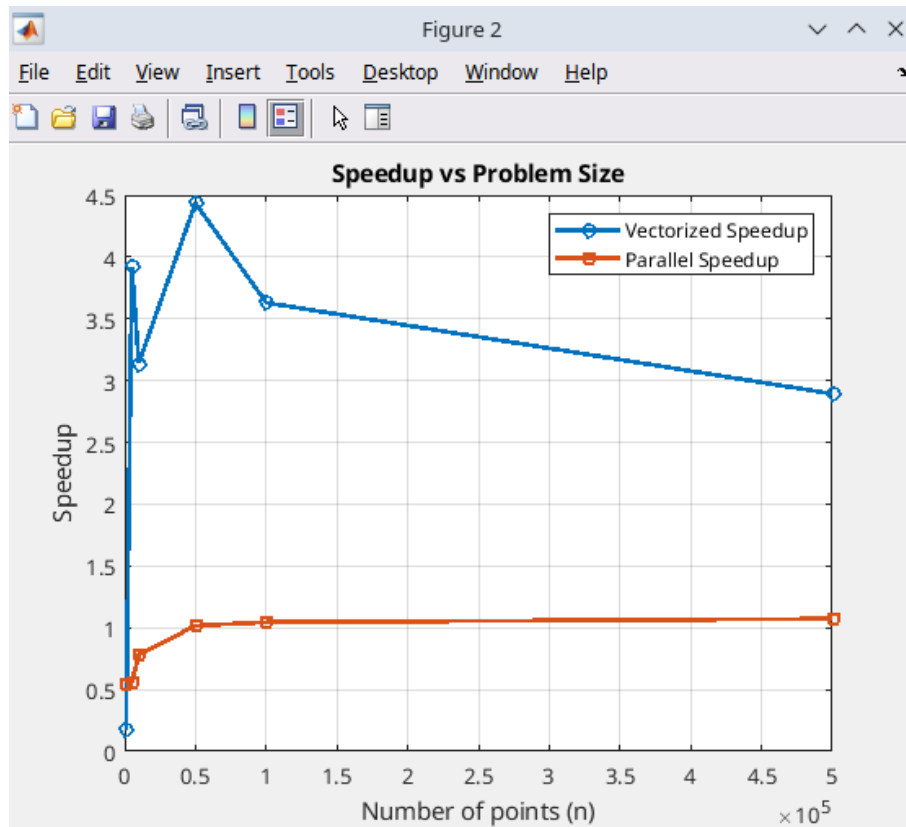


Figure 4: MATLAB - Speedup do Monte Carlo (Versões Vetorizada e Paralela vs. Serial)

3.3.2 Resultados MATLAB - Série de Leibniz

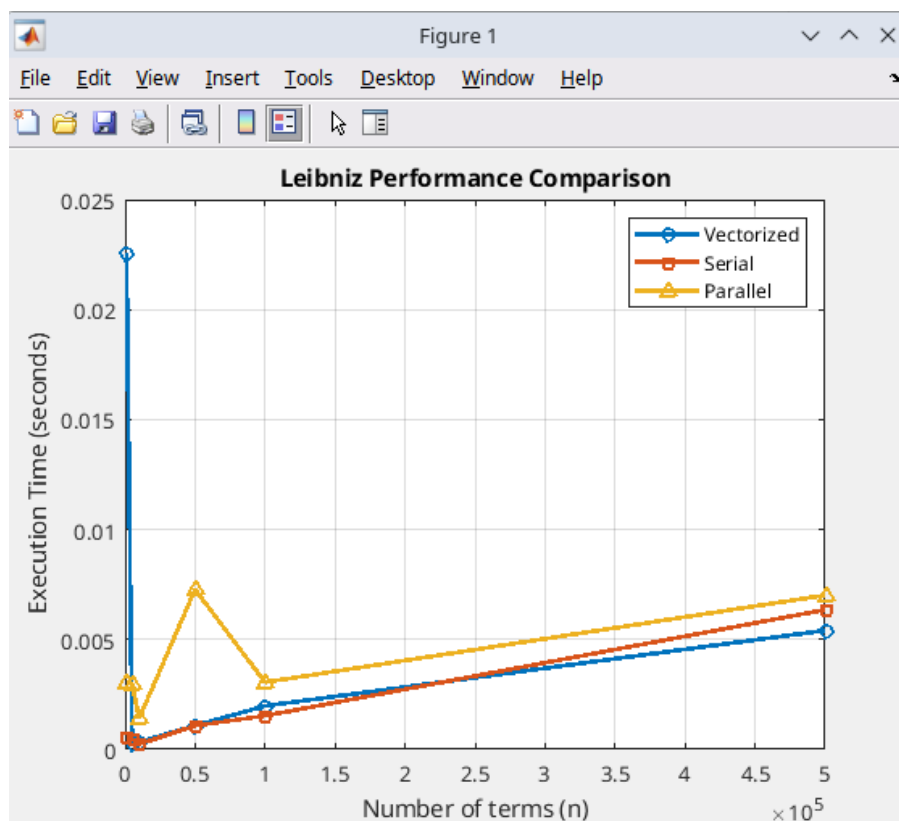


Figure 5: MATLAB - Comparação de Desempenho da Série de Leibniz (Tempo de Execução vs. Número de Termos)

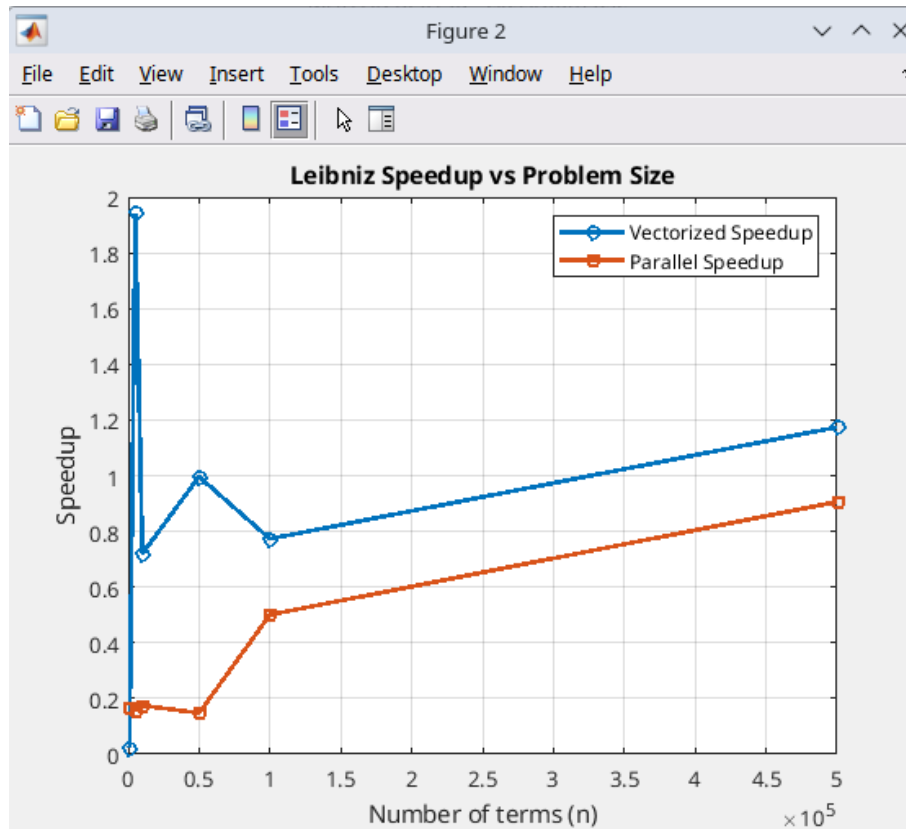


Figure 6: MATLAB - Speedup da Série de Leibniz (Versões Vetorizada e Paralela vs. Serial)

3.3.3 Resultados Python - Monte Carlo

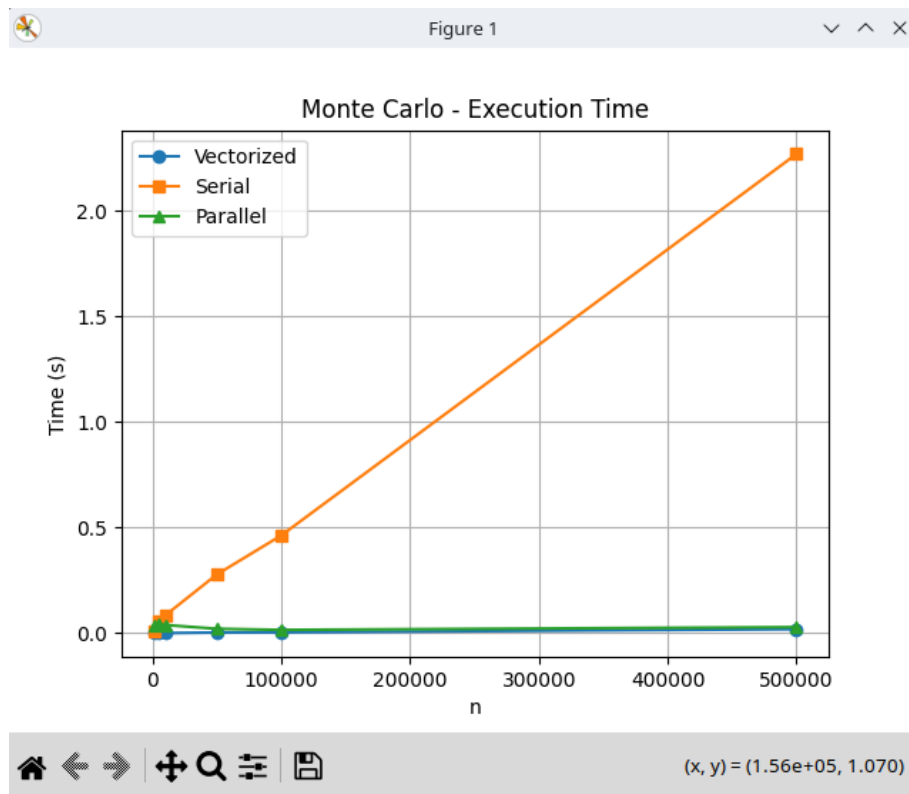


Figure 7: Python - Comparação de Tempo de Execução do Monte Carlo

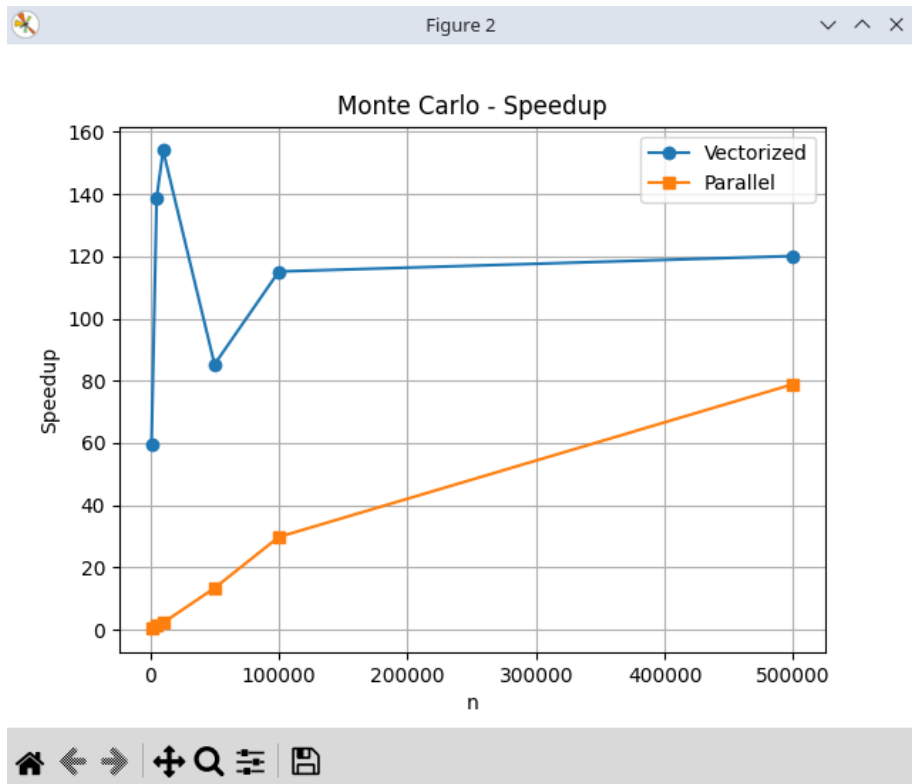


Figure 8: Python - Speedup do Monte Carlo (NumPy e Paralelo vs. Serial)

3.3.4 Resultados Python - Série de Leibniz

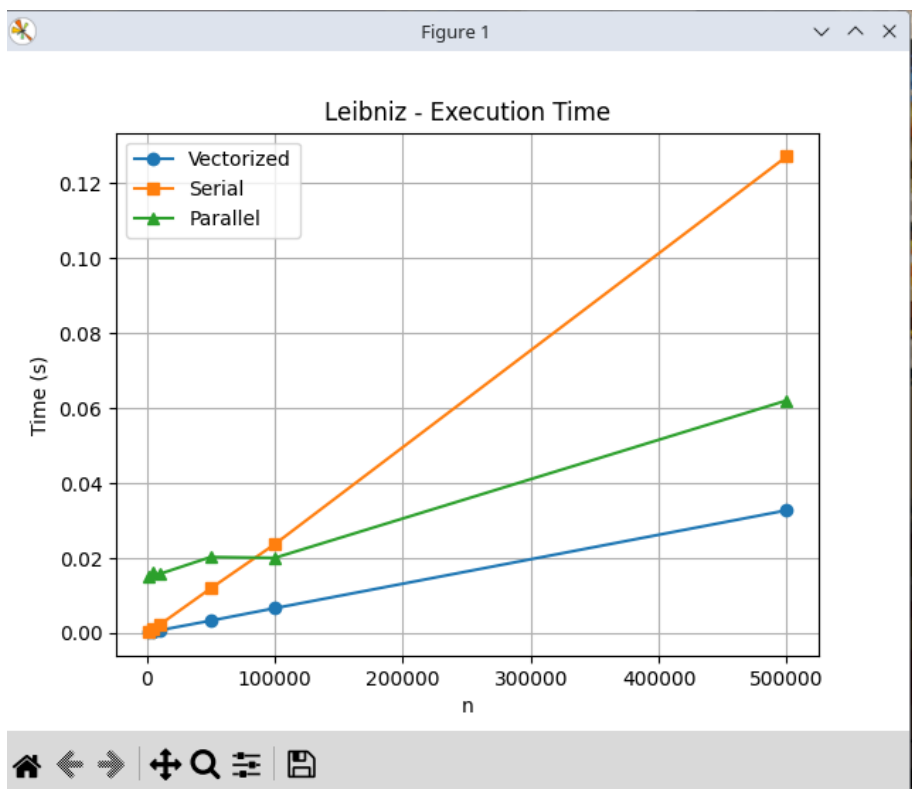


Figure 9: Python - Comparação de Tempo de Execução da Série de Leibniz

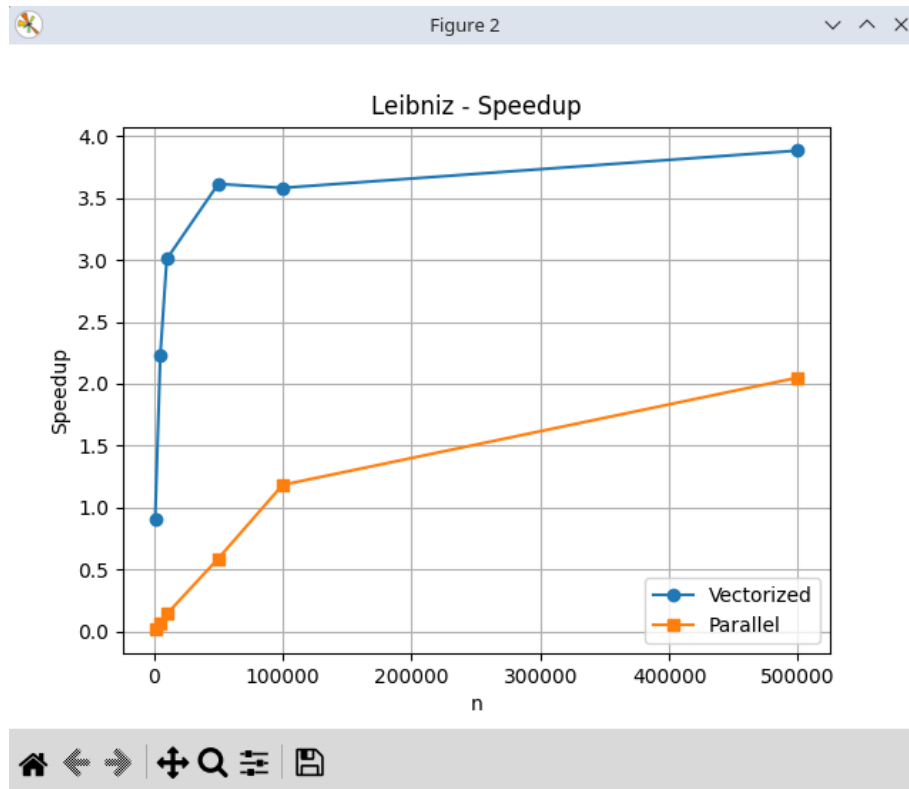


Figure 10: Python - Speedup da Série de Leibniz (NumPy e Paralelo vs. Serial)

3.4 Implementação — Método de Monte Carlo (Prática 1)

3.4.1 Versão Vetorizada

A versão vetorizada gera todos os n pontos de uma única vez como vetores, evitando laços explícitos e delegando ao motor interno do MATLAB a execução otimizada das operações em bloco:

3.4.2 Versão Serial

A versão serial percorre os n pontos individualmente num laço `for`, servindo como referência de desempenho:

3.4.3 Versão Paralela

A versão paralela substitui `for` por `parfor`, distribuindo as iterações entre os núcleos disponíveis via *Parallel Computing Toolbox*:

3.4.4 Cálculo do Speedup

Com os tempos registrados, os *speedups* são calculados tomando o tempo serial como base:

3.5 Implementação — Série de Leibniz (Prática 2)

3.5.1 Versão Vetorizada

Todos os termos da soma são computados simultaneamente por operações vetoriais:

Nota: a alocação do vetor k de 5×10^7 elementos demanda volume expressivo de memória RAM. Para valores de n muito elevados, o tempo de acesso à memória pode superar o tempo de computação, tornando esta versão a mais lenta das três.

3.5.2 Versão Serial

3.5.3 Versão Paralela

3.6 Simulação em Python

Com o objetivo de complementar a análise realizada em MATLAB, foram implementadas versões equivalentes dos algoritmos em Python, utilizando as bibliotecas `numpy` (vetorização) e `multiprocessing` (paralelização). Essa etapa permite avaliar diferenças de desempenho entre ambientes e explorar o comportamento das implementações em um ecossistema amplamente utilizado em computação científica.

3.6.1 Monte Carlo em Python

Versão Vetorizada (NumPy) A versão vetorizada utiliza operações sobre arrays do `numpy`, explorando otimizações internas em C:

Versão Paralela (multiprocessing) A paralelização é feita distribuindo o número total de amostras entre processos independentes:

3.6.2 Série de Leibniz em Python

Versão Vetorizada (NumPy)

Versão Serial

3.6.3 Análise Comparativa Inicial

De maneira geral, os resultados obtidos em Python seguem tendências semelhantes às observadas em MATLAB:

- O método de Monte Carlo apresenta bom desempenho em paralelo devido à independência das amostras;

- A vetorização com numpy tende a ser altamente eficiente, embora sujeita a limitações de memória para grandes valores de n ;
- A série de Leibniz não se beneficia significativamente de paralelização, devido ao baixo custo computacional por iteração;
- O *overhead* do multiprocessing em Python é mais elevado que o do parfor em MATLAB, impactando o speedup observado.

Essa comparação reforça que o desempenho não depende apenas do algoritmo, mas também da linguagem, das bibliotecas utilizadas e da forma como o paralelismo é implementado.

3.7 Resultados Experimentais

As Tabelas 6 e 7 resumizam os resultados obtidos para $n = 5 \times 10^7$. As Figuras 11 e 12 apresentam os gráficos de *speedup* em função de n , obtidos no ambiente MATLAB.

Table 6: Resultados do Método de Monte Carlo ($n = 5 \times 10^7$).

Implementação	$\hat{\pi}$	Erro (ε)	Tempo (s)	Speedup
Vetorizada	3.1415946400	2.00×10^{-6}	1.7892	8.87
Serial	3.1415830400	9.61×10^{-6}	15.8728	1.00
Paralela	3.1415885600	4.09×10^{-6}	7.4158	2.14

Table 7: Resultados da Série de Leibniz ($n = 5 \times 10^7$).

Implementação	$\hat{\pi}$	Erro (ε)	Tempo (s)	Speedup
Vetorizada	3.1415926836	3.00×10^{-8}	9.1561	0.41
Serial	3.1415926836	3.00×10^{-8}	3.7423	1.00
Paralela	3.1415926836	3.00×10^{-8}	5.6184	0.67

Figure 11: Speedup das versões vetorizada e paralela em relação à serial, em função de n — Método de Monte Carlo.

Figure 12: Speedup das versões vetorizada e paralela em relação à serial, em função de n — Série de Leibniz.

Figure 13: Ambiente MATLAB durante a execução dos experimentos.

4 Discussão dos Resultados

4.1 Precisão: Monte Carlo vs. Leibniz

Os resultados confirmam de forma inequívoca a superioridade da Série de Leibniz em termos de precisão para o mesmo número de iterações n . Para $n = 5 \times 10^7$, o erro do método de Monte Carlo na versão serial foi de 9.61×10^{-6} , enquanto a Série de Leibniz atingiu um erro de 3.00×10^{-8} em todas as implementações — uma diferença de mais de duas ordens de grandeza. Este resultado está perfeitamente alinhado com a análise teórica de convergência (Seção 2): Monte Carlo converge como $\mathcal{O}(1/\sqrt{n})$, enquanto a Série de Leibniz converge como $\mathcal{O}(1/n)$. O fator constante significativamente menor no limite de erro de Leibniz resulta em uma aproximação muito mais precisa para um n finito.

4.2 Desempenho: Monte Carlo

Para o método de Monte Carlo, a ordem de desempenho é inequívoca: $T_{\text{vet}} < T_{\text{par}} < T_{\text{ser}}$. A versão vetorizada, executando em 1.7892 segundos, é **8.87 vezes mais rápida** que a versão serial (15.8728 segundos). Este ganho dramático é atribuído à otimização interna do MATLAB para operações com arrays, que utiliza a Intel MKL (Math Kernel Library) para empregar instruções SIMD (como AVX) em computações altamente eficientes em nível C compilado.

A versão paralela (7.4158 segundos) alcançou um **speedup de 2.14x** em relação à serial. Este é um ganho significativo, mas notavelmente inferior ao speedup ideal de 4x esperado para um processador de 4 núcleos. Esta discrepância pode ser atribuída à sobrecarga inerente ao construto `parfor`: distribuição de tarefas, comunicação entre threads para a operação de redução (`count_par = count_par + 1`) e a latência da geração de números pseudoaleatórios em um contexto multi-thread. Apesar disso, um speedup acima de 2 confirma que o custo computacional por iteração (geração de números aleatórios + aritmética) é substancial o suficiente para superar essa sobrecarga para $n = 5 \times 10^7$.

4.3 Desempenho: Série de Leibniz

A Série de Leibniz apresenta um perfil de desempenho radicalmente diferente: $T_{\text{ser}} < T_{\text{par}} < T_{\text{vet}}$. A versão serial é a mais rápida, completando o cálculo em apenas 3.7423 segundos.

- **Falha da paralelização:** A versão paralela `parfor` levou 5.6184 segundos, resultando em um **speedup de apenas 0.67x**. Este é um exemplo clássico de uma carga de trabalho *não* adequada para paralelismo em nível de thread. O custo computacional de uma única iteração (uma divisão de ponto flutuante, uma adição e um incremento

de contador) é ofuscado pela sobrecarga de lançar, gerenciar e sincronizar threads paralelas. Como previsto pela Lei de Amdahl, quando a granularidade da porção paralelizável é muito fina, a sobrecarga domina, levando a uma perda líquida de desempenho.

- **Gargalo de memória na vetorização:** A versão vetorizada, apesar de sua vantagem teórica de velocidade, foi a mais lenta, com 9.1561 segundos (0.41x de speedup). Para $n = 5 \times 10^7$, o código cria dois vetores: `k` e `terms`. Cada um é um array de 50 milhões de números `double` (8 bytes), ocupando aproximadamente **400 MB cada**. As operações sobre esses vetores não são limitadas pela taxa de computação (FLOPS), mas pela **largura de banda da memória**. Os caches da CPU (L1, L2, L3) são muito pequenos para armazenar esses vetores, forçando transferências de dados constantes e lentas entre a CPU e a memória RAM principal. A versão serial, por outro lado, opera sobre um único valor escalar, reutilizando efetivamente os registradores da CPU e as linhas de cache, evitando assim este gargalo de largura de banda.

4.4 Análise de Speedup em Função de n

Os gráficos nas Figuras 4 e 6 fornecem uma visão detalhada da escalabilidade do desempenho.

- **Speedup do Monte Carlo:** O speedup da versão vetorizada aumenta rapidamente para n pequeno e torna-se dominante para n grande, refletindo a diminuição da sobrecarga da alocação inicial do array em relação ao trabalho computacional. O speedup paralelo também cresce com n , mas estabiliza em torno de 2x para os maiores valores. Este patamar é provavelmente devido à sobrecarga do parfor e a um certo nível de contenção na geração de números aleatórios que limita a escalabilidade além de dois núcleos para este tamanho específico de problema.
- **Speedup de Leibniz:** Os gráficos de speedup contam uma história de retornos decrescentes. O speedup vetorizado, inicialmente próximo de 1 para n pequeno, começa a cair abaixo de 1 para $n > 10^6$ e continua a se deteriorar à medida que n aumenta, demonstrando a severidade do gargalo de memória. O speedup paralelo permanece consistentemente abaixo de 1 para todos os valores exceto os menores, confirmando que o custo da iteração nunca é suficientemente alto para justificar a sobrecarga do paralelismo.

4.5 Arquitetura Paralela Mais Adequada

Com base nas evidências experimentais, as arquiteturas paralelas ótimas são:

- **Monte Carlo:** Dada sua alta granularidade e independência, este método é um candidato principal para arquiteturas **SIMD massivamente paralelas como GPUs**. O desempenho da versão vetorizada no MATLAB já sugere este potencial. Implementar isto em CUDA ou OpenCL provavelmente produziria speedups de várias centenas de vezes, pois os milhares de núcleos em uma GPU poderiam processar milhões de pontos em paralelo, superando em muito o limite de 4 núcleos da CPU.
- **Leibniz:** Este método não se beneficia da paralelização tradicional. A melhor abordagem é otimizar o código serial em nível de compilador (ex.: -O3 em C/C++) ou usar **instruções SIMD de forma vetorizada mas com consciência de cache**. A chave é usar uma técnica de processamento em blocos onde a soma é calculada em partes que cabem inteiramente no cache L2 ou L3 da CPU. Por exemplo, processar os n termos em blocos de 1000 ou 10.000 permitiria que as operações vetorizadas se beneficiassem do SIMD enquanto minimizam o tráfego de memória. Esta "vetorização com bloqueio de cache" teoricamente recuperaria a vantagem de velocidade perdida pela abordagem vetorizada ingênua.

5 Conclusões

A execução das Práticas 1 e 2 demonstrou com sucesso princípios fundamentais da computação de alto desempenho através das lentes de um problema matemático clássico.

1. **A Eficiência Algorítmica é Primordial:** O fator isolado mais importante determinando a precisão do resultado final foi a escolha do algoritmo matemático. A Série de Leibniz determinística, com sua convergência $\mathcal{O}(1/n)$, foi vastamente superior em precisão ao método estocástico de Monte Carlo ($\mathcal{O}(1/\sqrt{n})$) para o mesmo número de iterações. Isto ressalta o primeiro passo crítico em qualquer empreendimento de computação científica: selecionar o método numérico mais eficiente para o problema.
2. **Paralelismo Não é uma Panaceia:** Os experimentos forneceram uma validação prática da Lei de Amdahl. Para o problema de Monte Carlo, embaraçosamente paralelo, a paralelização produziu ganhos de desempenho tangíveis (2.14x de speedup). Para a Série de Leibniz de granularidade fina, a sobrecarga do paralelismo resultou em uma desaceleração líquida (0.67x de speedup). Isto demonstra que a granularidade do trabalho computacional é o determinante crucial para a viabilidade da paralelização. Uma análise cuidadosa da fração do programa que pode ser efetivamente paralelizada é essencial.
3. **Memória é o Novo Gargalo:** O desempenho da versão vetorizada da Série de Leibniz serviu como um lembrete contundente de que a computação moderna frequentemente

não é limitada por computação, mas **limitada por memória**. O uso ingênuo de grandes vetores para alcançar "simplicidade" e vetorização resultou completamente contraproducente. A imensa pressão sobre a largura de banda da memória fez com que a versão vetorizada se tornasse a implementação mais lenta, destacando que a otimização de desempenho deve considerar toda a hierarquia de memória — desde registradores até DRAM — não apenas as operações da CPU.

4. **Uma Visão Holística do Desempenho:** As práticas integraram com sucesso os três pilares da computação de alto desempenho: **design algorítmico** (convergência matemática), **estratégia de implementação** (serial, vetorizada, paralela) e **arquitetura de hardware** (núcleos da CPU, caches, largura de banda de memória). Uma solução de alto desempenho bem-sucedida requer uma compreensão e otimização balanceada de todas essas camadas.

Em conclusão, os resultados destas práticas transcendem o simples cálculo de π . Eles fornecem uma estrutura prática e baseada em experiência para analisar, implementar e avaliar criticamente o desempenho de algoritmos numéricos, reforçando os conceitos teóricos da disciplina com resultados concretos e mensuráveis. A implementação em Python, embora tenha mostrado tendências semelhantes, também destacou o impacto das sobrecargas em nível de linguagem e das otimizações de bibliotecas, fornecendo uma valiosa perspectiva multi-plataforma.

6 Referências Bibliográficas

References

- [1] BLOG CYBERINI. **Calculando o valor de Pi via Método de Monte Carlo**. Disponível em: <https://www.blogcyberini.com/2018/09/calculando-o-valor-de-pi-via-metodo-de-monte-carlo.html>. Acesso em: 25 mar. 2026.
- [2] UNIVERSIDADE DO MINHO. **Manipulação de vetores no Octave**. Disponível em: http://octave.di.uminho.pt/index.php/Manipula%C3%A7%C3%A3o_de_vetores. Acesso em: 25 mar. 2026.
- [3] WIKIPÉDIA. **Lei de Amdahl**. Disponível em: https://pt.wikipedia.org/wiki/Lei_de_Amdahl. Acesso em: 25 mar. 2026.
- [4] WIKIPÉDIA. **Fórmula de Leibniz para π** . Disponível em: https://pt.wikipedia.org/wiki/F%C3%B3rmula_de_Leibniz_para_%CF%80. Acesso em: 25 mar. 2026.

- [5] PYTHON SOFTWARE FOUNDATION. **multiprocessing — Process-based parallelism**. Disponível em: <https://docs.python.org/3/library/multiprocessing.html>. Acesso em: 25 mar. 2026.
- [6] PEDRINO, Emerson Carlos. **Roteiro da Prática 1: Cálculo de π por Monte Carlo**. São Carlos: UFSCar – Departamento de Computação, 2026. Disponível em: <https://ava.ufscar.br>. Acesso em: 25 mar. 2026.
- [7] PEDRINO, Emerson Carlos. **Roteiro da Prática 2: Cálculo de π por Leibniz**. São Carlos: UFSCar – Departamento de Computação, 2026. Disponível em: <https://ava.ufscar.br>. Acesso em: 25 mar. 2026.
- [8] PATTERSON, David A.; HENNESSY, John L. **Organização e Projeto de Computadores: a Interface Hardware/Software**. 5. ed. Rio de Janeiro: Elsevier, 2017.